

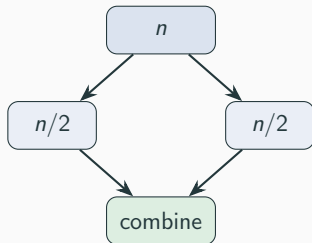
# Divide and Conquer, Part 1

Tiling Puzzle, Binary Search, Merge Sort, Solving Recurrences

---

# The Paradigm

- **Divide** a problem of size  $n$  into smaller sub-problems.
- **Conquer** them recursively.
- **Combine** their solutions into a solution for the original.
- Small  $n$  is solved directly: the **base case**.

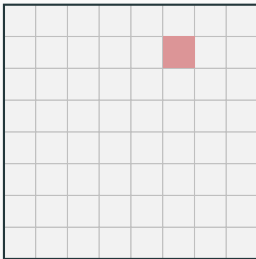


# Tiling Puzzle

---

# The Puzzle

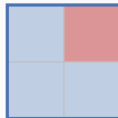
A  $2^n \times 2^n$  board has **one missing square**. Tile the rest with L-shaped trominoes, with no overlaps and nothing off the board.



8 × 8, one square missing

## Base Case: $2 \times 2$

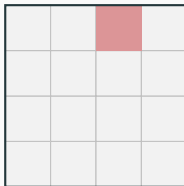
Trivial: orient the single tromino away from the missing square.



one tromino

## Exercise

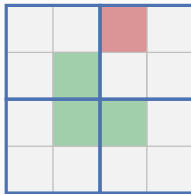
How do you reduce the  $4 \times 4$  case to the  $2 \times 2$  case?



Pause and think before the next slide.

# Solution

Split into four quadrants. One already has the missing square. Place **one tromino in the center** covering the other three.



Now every quadrant is a  $2 \times 2$  board with one square missing, which is the base case.

# The General Recursion

The same move works at every scale:

- Split the  $2^n \times 2^n$  board into four  $2^{n-1} \times 2^{n-1}$  quadrants.
- One quadrant holds the missing square.
- Drop one tromino in the center to punch a hole in each of the other three.
- Recurse on all four.

Every board of this shape can be tiled.

# Binary Search

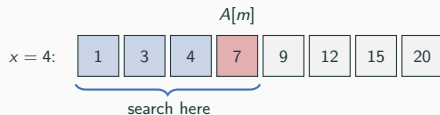
---

# Binary Search

**Input:** a **sorted** array  $A[1 \dots n]$  and a value  $x$ .

**Output:** whether  $x$  occurs in  $A$ .

Look at the middle  $m = \lfloor n/2 \rfloor$ . If  $x \leq A[m]$ , search  $A[1 \dots m]$ ; otherwise search  $A[m + 1 \dots n]$ .



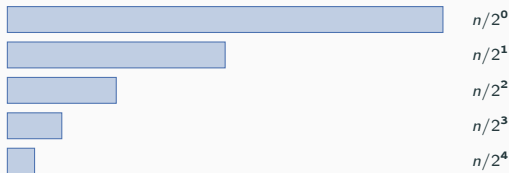
Each step **halves** the search range.

# Recursive Version

```
function binary_search(A, x, i, j)
  # Check if x is in A[i:j].
  # Return the index of x if found, otherwise nothing
  if i == j
    # Base case
    A[i] == x ? return i : return nothing
  else
    m = floor(Int, (i + j) / 2)
    if x <= A[m]
      return binary_search(A, x, i, m)
    else
      return binary_search(A, x, m + 1, j)
    end
  end
end
end
```

# Running Time

- The range starts at  $n$  and halves each level.
- Each level does  $O(1)$  non-recursive work.
- At level  $i$  the range is  $n/2^i$ ; setting  $n/2^i = 1$  gives  $i = \log_2 n$ .



**Total**

$$O(\log n) \times O(1) = O(\log n)$$

# Iterative Version

Often written iteratively, since it is simpler.

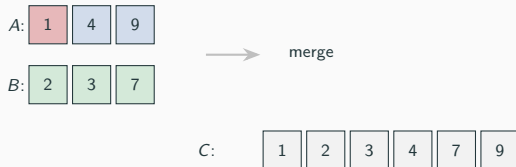
```
# Return the index of x in A if found, otherwise nothing
function binary_search(A, x)
    i, j = 1, length(A)
    while i < j
        m = floor(Int, (i + j) / 2)
        if x <= A[m]
            j = m
        else
            i = m + 1
        end
    end
    return (i <= length(A) && A[i] == x) ? i : -1
end
```

# Merge Sort

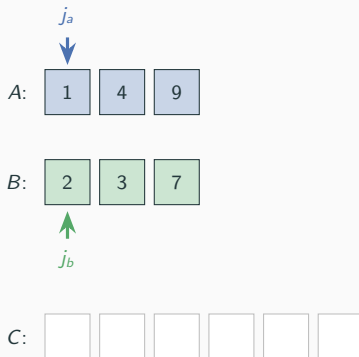
---

# Merging Two Sorted Arrays

Repeatedly take the smaller of the two front elements.

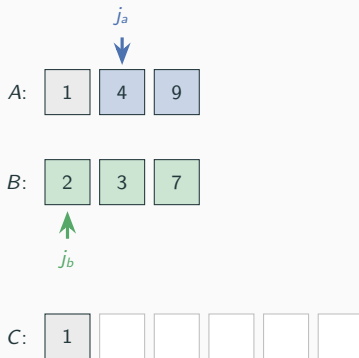


# Merging Step by Step



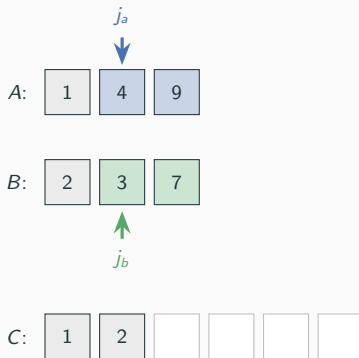
Compare 1 and 2. Take 1 from A.

# Merging Step by Step



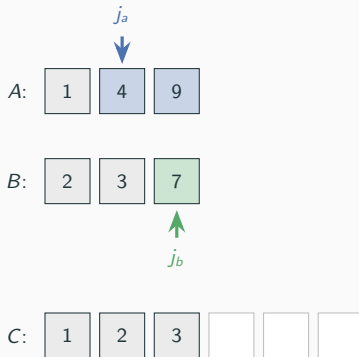
Compare 4 and 2. Take 2 from B.

# Merging Step by Step



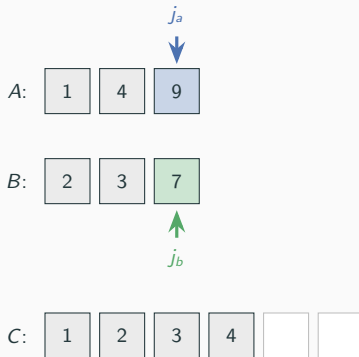
Compare 4 and 3. Take 3 from B.

# Merging Step by Step



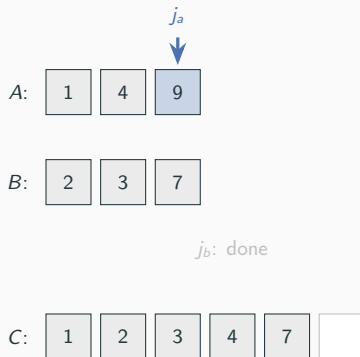
Compare 4 and 7. Take 4 from A.

# Merging Step by Step



Compare 9 and 7. Take 7 from B.

# Merging Step by Step



$B$  is exhausted. Copy the rest of  $A$ .

# Merging Step by Step

$j_a$ : done

A: 

|   |   |   |
|---|---|---|
| 1 | 4 | 9 |
|---|---|---|

B: 

|   |   |   |
|---|---|---|
| 2 | 3 | 7 |
|---|---|---|

$j_b$ : done

C: 

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 7 | 9 |
|---|---|---|---|---|---|

Done.

Each element is touched once, so merging costs  $O(n + m)$ .

# The Merge Procedure

```
function merge(A, B)
    n, m = length(A), length(B)
    C = Vector{eltype(A)}(undef, n + m)
    ja = jb = j = 1

    while ja <= n && jb <= m
        if A[ja] <= B[jb]
            C[j] = A[ja]; ja += 1
        else
            C[j] = B[jb]; jb += 1
        end
        j += 1
    end

    while ja <= n; C[j] = A[ja]; ja += 1; j += 1; end
    while jb <= m; C[j] = B[jb]; jb += 1; j += 1; end
    return C
end
```

# Merge Sort

Sort each half recursively, then merge.

```
function merge_sort(A)
    n = length(A)
    if n <= 1
        return A
    else
        mid = div(n, 2)
        L = @view A[1:mid]
        R = @view A[mid+1:end]

        sorted_L = merge_sort(L)
        sorted_R = merge_sort(R)

        return merge(sorted_L, sorted_R)
    end
end
```

## Merge Sort: Recurrence

Correctness is clear: after the recursive calls  $L$  and  $R$  are sorted, and merging preserves order.

Each level does  $O(n)$  non-recursive work and makes two calls of half the size:

$$T(n) = 2 T(n/2) + O(n)$$

We will show  $T(n) = O(n \log n)$ , and in fact  $\Theta(n \log n)$ .

Compare with selection sort's  $O(n^2)$ .

# Solving Recurrences

---

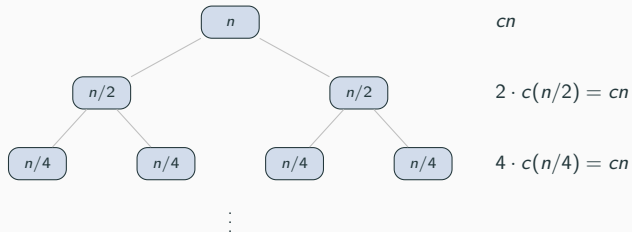
Draw one node per call. The running time is the **total non-recursive work** in the tree.

Three things can happen as you go down:

- Work per level stays the **same**: multiply by the number of levels.
- Work per level **shrinks** geometrically: the root dominates.
- Work per level **grows** geometrically: the leaves dominate.

## Case 1: Equal Work Per Level

$$T(n) = 2T(n/2) + cn \quad (\text{merge sort})$$



$$\log_2 n \text{ levels} \times cn \text{ per level} = O(n \log n)$$

## Case 2: Work Shrinks Geometrically

$$T(n) = T(n/10) + T(n/5) + cn^2$$

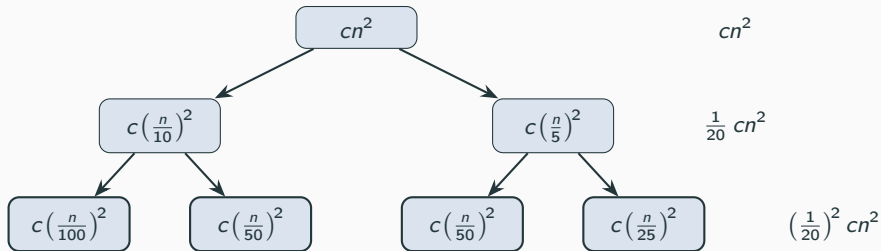
- Level 0:  $cn^2$
- Level 1:  $c \left( \frac{1}{100} + \frac{1}{25} \right) n^2 = c \left( \frac{1}{20} \right) n^2$
- Level 2:  $c \left( \frac{1}{20} \right)^2 n^2$

In general level  $i$  contributes  $c \left( \frac{1}{20} \right)^i n^2$ , so

$$\sum_{i=0}^{\log_5 n} c \left( \frac{1}{20} \right)^i n^2 \leq \sum_{i=0}^{\infty} c \left( \frac{1}{20} \right)^i n^2 = cn^2 \cdot \frac{1}{1 - \frac{1}{20}} = O(n^2).$$

## Case 2: Recursion Tree

$$T(n) = T(n/10) + T(n/5) + cn^2$$



Each level costs  $\frac{1}{20}$  of the level above  $\Rightarrow$  geometric shrinkage.

## Case 2: The Geometric Series

For  $r < 1$ :

$$\sum_{i=0}^{\infty} r^i = \frac{1}{1-r}, \quad \text{e.g.} \quad 1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \cdots = 2.$$

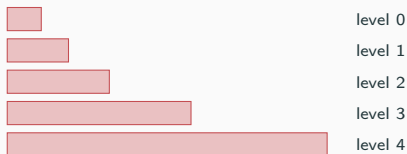


The **root** dominates: the total is a constant times the top level.

## Case 3: Work Grows Geometrically

$$T(n) = 7T(n/2) + cn^2 \quad (\text{Strassen's algorithm, later})$$

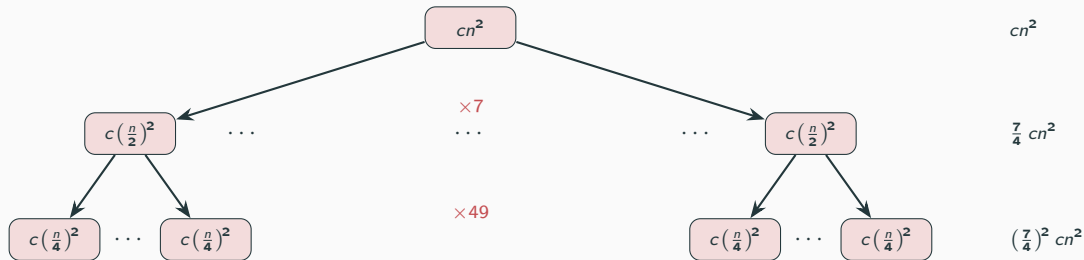
- Level 0:  $cn^2$
- Level 1:  $7c \left(\frac{n}{2}\right)^2 = \frac{7}{4}cn^2$
- Level 2:  $7^2c \left(\frac{n}{4}\right)^2 = \left(\frac{7}{4}\right)^2 cn^2$



Now the **deepest** level dominates.

### Case 3: Recursion Tree

$$T(n) = 7T(n/2) + cn^2$$



Level  $i$  costs  $\left(\frac{7}{4}\right)^i cn^2 \Rightarrow$  growing geometrically.

## Case 3: Summing Up

With at most  $\log_2 n$  levels,

$$\begin{aligned}\sum_{i=0}^{\log_2 n} \left(\frac{7}{4}\right)^i cn^2 &= c \cdot \frac{\left(\frac{7}{4}\right)^{\log_2 n + 1} - 1}{\frac{7}{4} - 1} n^2 \\ &= \Theta\left(n^2 \left(\frac{7}{4}\right)^{\log_2 n}\right) = \Theta\left(n^2 \cdot n^{\log_2(7/4)}\right).\end{aligned}$$

Since  $2 + \log_2(7/4) \approx 2.807$ :

$$T(n) = O(n^{2.807\dots})$$

# Master Theorem

If  $T(n) = a T(n/b) + O(n^d)$ , then

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a, \\ O(n^d \log n) & \text{if } d = \log_b a, \\ O(n^{\log_b a}) & \text{if } d < \log_b a. \end{cases}$$

The proof is a formalization of the three recursion-tree cases.

# Exercise

Solve each recurrence with the Master theorem.

- Binary search:  $T(n) = T(n/2) + O(1)$
- Merge sort:  $T(n) = 2T(n/2) + O(n)$
- Strassen:  $T(n) = 7T(n/2) + O(n^2)$

# Solution

|               | $a$ | $b$ | $d$ | result                                                  |
|---------------|-----|-----|-----|---------------------------------------------------------|
| Binary search | 1   | 2   | 0   | $d = \log_2 1 = 0 \Rightarrow O(\log n)$                |
| Merge sort    | 2   | 2   | 1   | $d = \log_2 2 = 1 \Rightarrow O(n \log n)$              |
| Strassen      | 7   | 2   | 2   | $d < \log_2 7 \approx 2.81 \Rightarrow O(n^{\log_2 7})$ |

The first two are the  $d = \log_b a$  case; Strassen is the leaf-dominated case.

## When It Does Not Apply

The Master theorem needs the exact form  $a T(n/b) + O(n^d)$ .

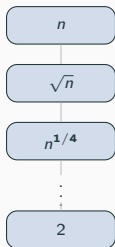
It says nothing about

$$T(n) = T(n/10) + T(n/5) + O(n^2),$$

because the two sub-problems have **different sizes**.

For those, fall back on the recursion tree.

Example:  $T(n) = T(\sqrt{n}) + O(1)$



Each level does  $O(1)$  work, so we just count levels  $k$ :

$$n^{1/2^k} = 2 \implies \frac{1}{2^k} \log_2 n = 1 \implies k = \log_2(\log_2 n).$$

Hence  $T(n) = O(\log \log n)$ .

**Example:**  $T(n) = T(n - 1) + O(n^2)$

The tree is a **path** of length  $n$ :  $n \rightarrow n - 1 \rightarrow n - 2 \rightarrow \dots \rightarrow 1$ .

Level  $i$  does  $c(n - i)^2$  work, so

$$T(n) = c(n^2 + (n - 1)^2 + \dots + 1^2) = \Theta(n^3),$$

using  $1^c + 2^c + \dots + n^c = \Theta(n^{c+1})$  from the previous chapter.

Shrinking by a constant *amount* is very different from shrinking by a constant *factor*.